

ROBUST KERNEL FISHER DISCRIMINANT ANALYSIS WITH WEIGHTED KERNELS

N. Louw

Department of Statistics and Actuarial Science, University of Stellenbosch,
South Africa

E-mail: *nlouw@sun.ac.za*

Key words: Kernel methods; classification; outliers; noisy data

Summary: Kernel Fisher discriminant analysis (KFDA) is a very popular kernel classification technique that performs well in situations where linear classifiers fail. The performance of the KFD classifier is however adversely affected by outliers or noise in the data. In this paper we propose a more robust KFD classifier by making use of weighted kernels. The performance of the proposed classifiers is compared to that of the KFD classifier with a Gaussian kernel in Monte Carlo simulation studies as well as on several benchmark data sets. Based on the results we conclude that the proposed weighted kernels are successful in achieving a lower error rate than the Gaussian kernel when used in a KFD classifier.

1. Introduction

Kernel Fisher discriminant analysis (KFDA) has recently become a very popular classification technique. It is a kernel based extension of Fisher linear discriminant analysis, and was developed by Mika et al. (1999). It entails transforming the data in the input space to a higher dimensional feature space via a non-linear transformation, and then performing linear discriminant analysis in this high (or even infinite) dimensional space. The resulting KFD classifier is linear in feature space, but corresponds to a non-linear classifier in the original input space. The classification performance of KFDA has been

AMS: 62H30

assessed in many papers, and is comparable to that of the support vector machine (SVM), arguably the most popular kernel classifier (cf. Mika et al., 1999 and Mika et al., 2001). However, the performance of these classifiers deteriorates in the presence of outliers and/or noise in the data. There are many contributions in the literature which deal with robust SVMs, but few dealing with robust KFDA. In this paper we propose a robust KFD classifier.

The paper is organised as follows. Section 2 introduces notation and provides a brief review of KFDA. In Section 3 the proposal for a more robust KFD classifier is discussed. Section 4 contains details of a Monte Carlo simulation study in which the classification performance of the proposed classifier is assessed. In Section 5 results obtained on several benchmark data sets are given. Conclusions and directions for further research are formulated in Section 6.

2. Kernel Fisher discriminant analysis

Consider a two-group classification problem where we observe a binary response variable $Y \in \{-1, +1\}$, as well as input variables $X_1, X_2, \dots, X_p$. We observe a total of n data cases: n_1 cases from population 1 ($Y = +1$) and n_2 cases from population 2 ($Y = -1$). We write $J_1 = \{1, 2, \dots, n_1\}$ for the set of indices corresponding to the cases from population 1, and $J_2 = \{n_1 + 1, n_1 + 2, \dots, n\}$ for those from population 2. The resulting training data set is denoted by $\mathcal{T} = \{(\mathbf{x}_i, y_i), i = 1, 2, \dots, n\}$. The objective is to use $\mathcal{T}$ to find a function, $f_{\mathcal{T}}(\mathbf{x})$, so that $\text{sign}\{f_{\mathcal{T}}(\mathbf{x})\}$ can be used to assign a new case with observed values of the classification variables in the vector $\mathbf{x}$ to one of the two populations. We refer to $\text{sign}\{f_{\mathcal{T}}(\mathbf{x})\}$ as a *classifier*. Conceptually KFDA entails transformation of the $\mathbf{x}_i \in \mathcal{R}^p$ to a

high-dimensional *feature space*, followed by application of the usual LDA procedure in feature space. Although the classifier resulting from this will be linear in feature space, it corresponds to a non-linear classifier in input space. Let $\Phi(\cdot)$ denote the transformation from input to feature space, with $\Phi(\mathbf{x})$ a transformed input vector, also called a feature vector. Since the dimension of the feature space is typically very high and can even be infinite, performing calculations in this space is problematic. In KFDA, as in other kernel based procedures, this difficulty is overcome by application of the *kernel trick*, which entails replacing inner products between feature vectors by a kernel function operating on pairs of input vectors: $\langle \Phi(\mathbf{x}_j), \Phi(\mathbf{x}_k) \rangle = K(\mathbf{x}_j, \mathbf{x}_k)$. The important implication of this is that, if the feature vectors appear in the algorithm used to construct a classifier in feature space only in the form of inner products, it obviates explicit specification of $\Phi(\cdot)$ as well as calculations in feature space. In practice this is implemented by specifying and using a kernel without considering the implied feature mapping $\Phi(\cdot)$ or the space which it induces. A very popular kernel is the Gaussian kernel, defined by $K(\mathbf{x}_k, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_k - \mathbf{x}_j\|^2\right)$, where $\gamma > 0$ is a hyperparameter which has to be specified beforehand or determined from the data. The KFD classifier (cf. Mika et al., 1999) is given by

$$\text{sign} \left\{ b + \sum_{i=1}^n \tilde{\alpha}_i K(\mathbf{x}_i, \mathbf{x}) \right\}. \quad (1)$$

To determine the coefficients $\tilde{\alpha}_i, i = 1, \dots, n$, in (1), let α be an n -vector with elements $\alpha_1, \alpha_2, \dots, \alpha_n$. Then $\tilde{\alpha}$ maximises the Rayleigh coefficient

$$J(\alpha) = \frac{\alpha' \mathbf{M} \alpha}{\alpha' \mathbf{N} \alpha}. \quad (2)$$

In (2), $\mathbf{M} = (\mathbf{M}_1 - \mathbf{M}_2)(\mathbf{M}_1 - \mathbf{M}_2)'$ and $\mathbf{N} = \mathbf{K}'\mathbf{K} - n_1\mathbf{M}_1\mathbf{M}_1' - n_2\mathbf{M}_2\mathbf{M}_2'$, where the $n \times n$ kernel matrix (also called the Gram matrix) $\mathbf{K}$ has elements

$K(\mathbf{x}_k, \mathbf{x}_j)$, $j, k = 1, \dots, n$, and the n elements of $\mathbf{M}_1$ are given by $\frac{1}{n_1} \sum_{j \in J_1} K(\mathbf{x}_k, \mathbf{x}_j)$, $k = 1, \dots, n$, with a similar expression for $\mathbf{M}_2$. To solve for $\tilde{\alpha}$, singularity of the N matrix is overcome by regularisation: N is replaced by a matrix $N_\lambda = N + \lambda \mathbf{I}$, for some (small) positive scalar λ . This yields the solution $\tilde{\alpha} = N_\lambda^{-1} (\mathbf{M}_1 - \mathbf{M}_2)$, which can be used in (1). Obviously the hyperparameter λ has to be specified or determined from the data. A popular choice for the intercept in (1) is

$$b = \frac{1}{2} (\mathbf{M}_2' N_\lambda^{-1} \mathbf{M}_2 - \mathbf{M}_1' N_\lambda^{-1} \mathbf{M}_1) + \log(n_1/n_2).$$

3. Robust kernel Fisher discriminant analysis

Many papers on robust SVMs have been published. A detailed discussion of these proposals falls outside the scope of this paper. In some papers, such as Song et al. (2002) and Trafalis and Gilbert (2005), robust optimisation problems that can be solved via linear or convex programming are formulated. Herbrich and Weston (1999) propose adaptive margin SVMs, while Xu et al. (2006) consider robust SVM training that is based on identifying and eliminating outliers. Liao and Lai (2007) and Chen (2004) propose the use of robust kernels in the standard SVM algorithm.

In a KFDA context, Roth (2006) develops a one-class kernel Fisher discriminant classifier (OC-KFD) for outlier detection. Roth (2006) establishes a relation between kernelised one-class classification and Gaussian density estimation in feature space. This relation is then used to iteratively identify outliers by measuring deviations from normality. Kim et al. (2006) develop robust *linear* Fisher discriminant analysis by incorporating a model of data uncertainty in the classification problem and by optimising the worst-case

scenario under this model. They show how this can be extended to robust KFDA, and mention that the problem can be formulated as a quadratically constrained quadratic program. They do not pursue this any further and no numerical results are given for the robust KFDA (only for the robust LDA).

Li et al. (2007) propose a probabilistic kernel Fisher method for classification in the presence of class noise (mislabelled cases). Instead of using the noisy class labels directly, the posterior probability of a case belonging to a specific class (given the input variables and the observed label) is estimated. These probabilities are used to obtain probabilistically weighted versions of the matrices $\mathbf{M}$ and $\mathbf{N}$ in the Rayleigh coefficient (2), which is then maximised to obtain the probabilistic KFD classifier. Li et al. (2007) describe a method for estimating the posterior probabilities via the EM algorithm. The performance of the probabilistic KFD classifier is compared to that of other classification procedures (among which quadratic discriminant analysis, standard KFDA and k-nearest neighbours). The procedure performs well. The paper specifically focuses on label noise, and does not deal with input variable noise or other types of outliers.

Liang et al. (2006) develop a method called robust kernel discriminant analysis with the specific aim of dimensionality reduction. Their proposed kernel based algorithm is applied to high dimensional data to find a transformation to a lower dimension, and nearest neighbour classification is then used. They do not perform KFDA for classification purposes.

Louw and Lamont (2009) propose a KFDA procedure using a robust Mahalanobis kernel, $K(\mathbf{x}_k, \mathbf{x}_j) = \exp(-\gamma(\mathbf{x}_k - \mathbf{x}_j)' \mathbf{S}_r^{-1}(\mathbf{x}_k - \mathbf{x}_j))$, $j, k = 1, \dots, n$, where $\mathbf{S}_r$ denotes a robust covariance estimator. They use the minimum covariance determinant estimator (cf. Rousseeuw, 1985) and the nearest neighbour variance estimator (cf. Wang and Raftery, 2002). The KFD

classifier using the robust Mahalanobis kernel performed well when compared to the KFD classifiers using the Gaussian and standard Mahalanobis kernels. A disadvantage is that this classifier cannot be used for so-called wide data, i.e. data with more variables than observations. In this paper we propose a procedure that can also be used for wide data.

As mentioned in Section 2, the Gaussian or radial basis function (RBF) kernel, $K(\mathbf{x}_k, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_k - \mathbf{x}_j\|^2\right)$, $j, k = 1, \dots, n$, is often used in KFDA and other kernel methods. When there are outliers in the training data, i.e. cases that are in a certain sense “different” or “far away” from the rest of the cases in a specific group, such cases will typically result in an unusually large value of $\|\mathbf{x}_k - \mathbf{x}_j\|^2$, and thus an unusually small value of $K(\mathbf{x}_k, \mathbf{x}_j)$. Noise variables will have a similar effect. Since the KFD algorithm operates on the entries $K(\mathbf{x}_k, \mathbf{x}_j)$ in the kernel matrix, limiting the effect of outliers on these entries should reduce the effect of such outliers on the KFD classifier. This could be achieved in different ways, amongst others by replacing the quadratic term in the Gaussian kernel with a more robust form, such as an absolute value or Huber function.

However, our proposal is to modify the Gaussian kernel by using different weight functions to reduce the effect of outliers and noise. We therefore propose the weighted kernel

$$K_w(\mathbf{x}_k, \mathbf{x}_j) = \exp\left(-\gamma \sum_{i=1}^p w_i^{kj} (x_{ki} - x_{ji})^2\right), \quad j, k = 1, \dots, n.$$

The weights w_i^{kj} are determined from the training data and we consider different ways of calculating these weights. Writing $z_i^{kj} = |x_{ki} - x_{ji}|$, we propose using the following weight functions:

- $w_{1i}^{kj} = \left(1 + \left(z_i^{kj}/\delta\right)^2\right)^{-1}$

- $w_{2i}^{kj} = 1$ for $z_i^{kj} \leq \delta$; $w_{2i}^{kj} = \delta/z_i^{kj}$ for $z_i^{kj} > \delta$
- $w_{3i}^{kj} = 1$ for $z_i^{kj} \leq \delta$; $w_{3i}^{kj} = \left(\delta/z_i^{kj}\right)^2$ for $z_i^{kj} > \delta$
- $w_{4i}^{kj} = \left(1 + \frac{1}{2} \left(z_i^{kj}/\delta\right)^2\right)^{-1}$

These weight functions are often used in robust methods, cf. Holland and Welsch (1977), Chen (2004) and Geman and McClure (1987). Graphs of the weight functions (for $\delta = 1$) appear in Figure 1.

As is clear from the graphs, the different weight functions apply weights that decrease with increasing absolute difference between two cases. This will result in large differences having less effect, and will therefore decrease the effect of outliers and noise. Since the weighting is done per variable, provision is made for the fact that cases are not necessarily outliers with respect to all the observed variables, but may be outliers only with respect to some variables. The value of δ has to be specified or determined from the data. In the analyses that follow in Section 4 (Monte Carlo simulation study) and Section 5 (benchmark data sets), the value of δ is determined from the data by means of cross-validation. Details will be provided in the relevant sections.

The proposed weighting scheme differs from that suggested by Bo et al. (2006), where feature scaling kernels, amongst which feature scaled Gaussian kernels, are used in KFDA. In feature scaling kernels weights are also used, but these weights reflect the importance of each variable (feature) and do not attempt to reduce the effect of outliers. The weights are also determined in

a different way to the weights proposed in this paper.

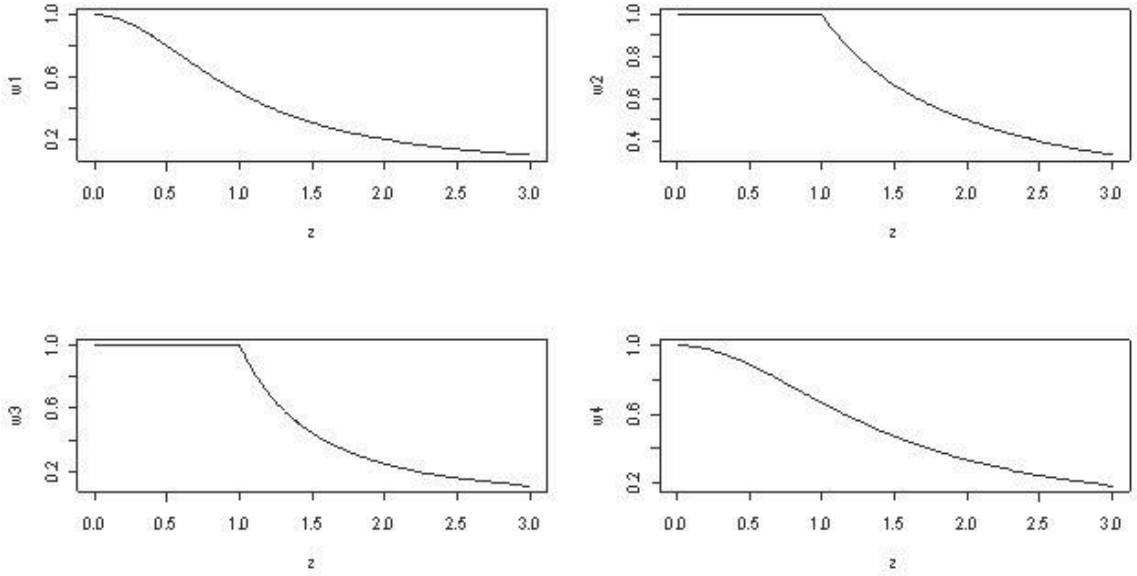


Figure 1. Weight functions for use in robust kernels (for $\delta = 1$).

When considering different kernels, measures which evaluate the “goodness” of the kernels for a classification task can provide an indication of the performance of the resulting classifiers. Various such measures have been proposed in the literature. Cristianini et al. (2002) proposed the kernel target alignment (KTA)

$$KTA = \frac{\langle \mathbf{K}, \mathbf{y}\mathbf{y}' \rangle_F}{\sqrt{\langle \mathbf{K}, \mathbf{K} \rangle_F \langle \mathbf{y}\mathbf{y}', \mathbf{y}\mathbf{y}' \rangle_F}},$$

where $\langle \mathbf{R}, \mathbf{S} \rangle_F = \text{trace}(\mathbf{R}\mathbf{S})$, is the Frobenius inner product, and $\mathbf{y}$ the response vector with classes coded as $+1$ and -1 , and $\mathbf{y}\mathbf{y}'$ is considered as the “ideal” Gram matrix. Cristianini et al. (2002) state that a large value of the alignment is desirable since the generalisation error of a kernel classifier using $\mathbf{K}$ improves with an increase in alignment.

Nguyen and Ho (2008) claim that a high value of the kernel target alignment is only a sufficient but not a necessary condition for a kernel matrix to perform well. They therefore propose a feature space based kernel matrix evaluation measure (FSM), defined as the ratio of the total within class standard deviation *in the direction between the class centres* to the distance between the class centres:

$$FSM = \frac{std_1 + std_2}{\|\mathbf{c}_1 - \mathbf{c}_2\|}. \quad (3)$$

In (3), $\mathbf{c}_i = \frac{1}{n_i} \sum_{j \in J_i} \Phi(\mathbf{x}_j)$ denotes the i^{th} group mean (centre), calculated in feature space. The variance of the observations in the i^{th} group in the direction between the class centres is denoted by

$$std_i^2 = \frac{1}{n_i - 1} \sum_{j \in J_i} \langle (\Phi(\mathbf{x}_j) - \mathbf{c}_i), \mathbf{u} \rangle^2, \quad i = 1, 2, \quad (4)$$

where $\mathbf{u} = (\mathbf{c}_1 - \mathbf{c}_2) / \|\mathbf{c}_1 - \mathbf{c}_2\|$ is the unit vector in the direction between the class centres.

Using the kernel trick, the distance between the group centres can be written as

$$\|\mathbf{c}_1 - \mathbf{c}_2\| = \sqrt{\frac{1}{n_1^2} \sum_{j \in J_1} \sum_{k \in J_1} K(\mathbf{x}_j, \mathbf{x}_k) + \frac{1}{n_2^2} \sum_{j \in J_2} \sum_{k \in J_2} K(\mathbf{x}_j, \mathbf{x}_k) - \frac{2}{n_1 n_2} \sum_{j \in J_1} \sum_{k \in J_2} K(\mathbf{x}_j, \mathbf{x}_k)}.$$

The kernel trick can also be used to obtain expressions for the standard deviations. Define

$$\tilde{K}_1(\mathbf{x}_j, \mathbf{x}_k) = K(\mathbf{x}_j, \mathbf{x}_k) - \frac{1}{n_1} \sum_{l \in J_1} K(\mathbf{x}_l, \mathbf{x}_k).$$

Then

$$std_1^2 = \frac{\frac{1}{(n_1-1)} \sum_{j \in J_1} \left\{ \frac{1}{n_2} \sum_{k \in J_2} \tilde{K}_1(\mathbf{x}_j, \mathbf{x}_k) - \frac{1}{n_1} \sum_{k \in J_1} \tilde{K}_1(\mathbf{x}_j, \mathbf{x}_k) \right\}}{\frac{1}{n_1^2} \sum_{j \in J_1} \sum_{k \in J_1} K(\mathbf{x}_j, \mathbf{x}_k) + \frac{1}{n_2^2} \sum_{j \in J_2} \sum_{k \in J_2} K(\mathbf{x}_j, \mathbf{x}_k) - \frac{2}{n_1 n_2} \sum_{j \in J_1} \sum_{k \in J_2} K(\mathbf{x}_j, \mathbf{x}_k)}.$$

A similar expression can be obtained for std_2^2 . A small value of the FSM is desirable. It is important to emphasise that the variance defined in (4) differ from the usual within-group variance of the i^{th} group,

$s_i^2 = \frac{1}{n_i-1} \sum_{j \in J_i} \|\Phi(\mathbf{x}_j) - \mathbf{c}_i\|^2$. In the latter, the total variation (in feature space) within each group is calculated, but the direction in which the variation occurs is not taken into account. In (4), variation in the direction between the two class centres (in feature space) is calculated. In a classification problem, it is specifically large variation in this direction that can lead to large error rates. The graphs in Figure 2 (in two dimensions) illustrate this concept.

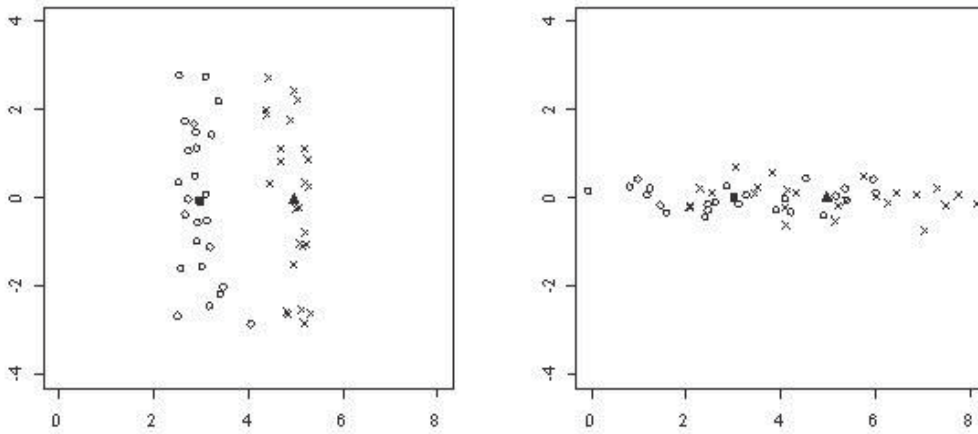


Figure 2. Scatterplots of bivariate data with variation in different directions.

The total within group variation (not taking the direction of the variation into account) as well as the distance between the group centres (indicated by a solid square and triangle respectively) are more or less the same for the data sets depicted in the left and right panels of Figure 2. However, in the left panel

the within group variation in the direction between the group centres is small, while there is large within group variation in the direction perpendicular to the direction between the group centres. In the right panel the situation is reversed. Clearly, a classifier should perform much better in the scenario depicted on the left than on the right and the *FSM* criterion will reflect this by having a smaller value for the data in the left panel than for the data in the right panel.

Based on theoretical arguments illustrating shortcomings of the *KTA* and empirical evidence obtained on synthetic data as well as on benchmark data sets, Nguyen and Ho (2008) concluded that the *FSM* “is more closely correlated to the goodness of a kernel matrix” than the *KTA*. In the comparison of the proposed weighted kernels and the Gaussian kernel, we calculated both these measures and reached a similar conclusion. Details are provided in Section 4.

4. Monte Carlo simulation study

To compare the performance of the KFD classifiers obtained when using the proposed weighted kernels and the Gaussian kernel, a detailed Monte Carlo simulation study was done. Various scenarios were considered: data without outliers or noise, data with different proportions of outliers (10%, 20% and 30%) but without noise, and data with outliers (10%, 20% and 30%) as well as noise were included in the simulation study. Other factors varied were:

- The distributions from which the input vectors $\mathbf{x}_i$ were generated. We used normal and lognormal distributions.
- The training sample sizes.
- The number p of input variables and the number of variables (denoted by p_r) with respect to which the groups differ (referred to as relevant variables in the remainder of the paper). The remaining variables were noise variables (denoted by p_n).

- The correlations between the input variables. We assumed equal correlations between all pairs of relevant variables. Writing ρ for this common correlation, we investigated $\rho = 0$ (giving uncorrelated input variables), $\rho = -0.1$ (yielding equi-correlated input variables with a small negative correlation) and $\rho = 0.5$ (yielding equi-correlated input variables which are fairly strongly positively correlated). Noise variables were always uncorrelated with all other variables. Naturally, it is desirable to investigate other correlation structures as well, and this is one of the topics for further research.
- The parameters with respect to which the two populations differed. Cases which were investigated corresponded to populations having identical covariance matrices but differing with respect to mean vectors, and populations having identical mean vectors but differing with respect to covariance matrices. We refer to these as “differences in location” and “differences in spread” respectively. Investigating cases where the populations vary with respect to both location and spread simultaneously, is also a topic for further research.

Details of the exact parameter configurations are given in each section.

In each case, training data were generated from the relevant distribution using the specified parameter configurations. The training data were standardised so that each input variable has zero mean and unit variance. This is standard practice in the application of kernel methods, and often leads to improved results (cf. Schölkopf and Smola, 2002). For each of the four weighted kernels, 5-fold cross-validation was performed to determine the values of δ (for use in the specific weight function) and the hyperparameter λ , used for regularisation when computing the KFD classifier. We searched over a grid of values: for δ , 15 values evenly spaced over the interval $[0.1, 1.5]$ were considered and for λ we considered 15 values spaced on a log-scale over the interval $[e^{-11}, e^3]$. Specifying a value for the kernel hyperparameter γ is an important problem, (cf. for example Wang et al., 2003). Cross-validation is a viable albeit time-consuming option. Empirical evidence obtained in other

related studies of kernel methods suggest that $\gamma = 1/p$ is often a reasonable choice (cf. Oosthuizen, 2008 and Lamont, 2008). We therefore use $\gamma = 1/p$, although specifying γ by means of cross-validation or some other adaptive technique deserves further attention. For the Gaussian kernel, we searched over the same range as before for λ . Using a fixed value of $\gamma = 1/p$ here may put the Gaussian kernel at an unfair disadvantage compared to the weighted kernels, since it could be argued that more “fine tuning” was done for the weighted kernels by optimising over two parameters. For the Gaussian kernel we therefore determined γ by searching over 12 values: the value $\gamma = 1/p$ as well as 11 values in the interval $[10^{-5}, 10^5]$. The optimal hyperparameter values were then used and the KFD classifiers using the 5 kernels were obtained. To estimate the error rates, an independent test data set containing 1000 cases per group was generated from the same distribution as the training data and classified by the five KFD classifiers. This was repeated 100 times to estimate the average error rate and standard error of each classifier. We also calculated the two kernel evaluation measures, the *KTA* and *FSM* discussed in Section 3, and calculated the average of each measure.

Configuration 1

We firstly consider lognormal data with location difference. We took $p_r = 5$ and $p_n = 0$. Data were simulated from normal distributions with zero means and equal covariance matrices, $\Sigma_1 = \Sigma_2 = \Sigma^{(\rho)}$, where $\Sigma^{(\rho)}$ is a matrix with diagonal elements $\sigma_{ii} = 1$, $i = 1, \dots, p_r$, and off-diagonal elements $\sigma_{ij} = \rho$, $\forall i \neq j$, $i, j = 1, \dots, p_r$. The data were then transformed to lognormal data and mean vectors $\mu_1 = 2\mathbf{e}_5$ and $\mu_2 = 3\mathbf{e}_5$, where $\mathbf{e}_k$ denotes a k -vector with all the elements equal to 1, were added. Data without outliers as well as data with 10%, 20% and 30% outliers were considered. For outliers we

took $\mu_1 = 4\mathbf{e}_5$ and $\mu_2 = \mathbf{e}_5$. Sample sizes (including outliers if present) of $n_1 = n_2 = 10$ were used. The average test errors (with standard errors in brackets) appear in Table 1. For each configuration the lowest test error appears in bold face.

Table 1 Mean and standard error of test errors for lognormal data (location difference).

% Outliers	ρ	Gaussian	w_1	w_2	w_3	w_4
0	-0.1	.060(.0019)	.017(.0011)	.022(.0014)	.046(.0028)	.026(.0019)
	0	.081(.0021)	.031(.0016)	.032(.0017)	.062(.0038)	.040(.0025)
	0.5	.109(.0027)	.092(.0027)	.099(.0030)	.108(.0030)	.097(.0029)
10	-0.1	.096(.0029)	.022(.0014)	.030(.0033)	.059(.0034)	.027(.0021)
	0	.110(.0028)	.035(.0025)	.036(.0021)	.069(.0028)	.035(.0024)
	0.5	.155(.0033)	.096(.0026)	.106(.0033)	.131(.0036)	.102(.0031)
20	-0.1	.110(.0031)	.037(.0026)	.048(.0040)	.086(.0040)	.042(.0034)
	0	.126(.0035)	.041(.0025)	.057(.0038)	.107(.0045)	.049(.0036)
	0.5	.172(.0046)	.119(.0039)	.114(.0042)	.154(.0047)	.106(.0042)
30	-0.1	.140(.0040)	.067(.0036)	.065(.0037)	.098(.0038)	.068(.0039)
	0	.149(.0040)	.074(.0033)	.085(.0037)	.113(.0036)	.090(.0038)
	0.5	.189(.0043)	.136(.0037)	.135(.0049)	.158(.0043)	.132(.0035)

It is clear from Table 1 that the performance of the KFD classifier using the Gaussian kernel is improved by the proposed weighting. In general the kernel with weight w_1 performs best. The Gaussian kernel always resulted in the highest error rate.

To investigate the better performance of the weighted kernels when compared to the Gaussian kernel, we calculated the kernel evaluation measures discussed in Section 3: the *KTA* (Table 2) and the *FSM* (Table 3).

Firstly, consider the *KTA* values in Table 2. A large value of this measure is desirable and for each configuration the highest value appears in bold face. The results seem contradictory: the Gaussian kernel, which yielded the worst

estimated error rate in all the cases in Table 1, has the highest KTA value in all cases. These results are in line with the findings of Nguyen and Ho (2008) that a large value of the KTA does not necessarily imply that a kernel will perform well in a classification task, and precisely why Nguyen and Ho (2008) proposed the FSM as a measure rather than the KTA .

Table 2 Kernel target alignment values for lognormal data (location difference).

% Outliers	ρ	Gaussian	w_1	w_2	w_3	w_4
0	-0.1	0.378	0.192	0.277	0.305	0.236
	0	0.362	0.186	0.253	0.287	0.230
	0.5	0.333	0.158	0.198	0.253	0.18
10	-0.1	0.264	0.068	0.091	0.204	0.063
	0	0.249	0.051	0.105	0.190	0.071
	0.5	0.219	0.060	0.107	0.168	0.077
20	-0.1	0.177	0.034	0.061	0.136	0.047
	0	0.162	0.025	0.056	0.122	0.038
	0.5	0.147	0.038	0.066	0.114	0.038
30	-0.1	0.115	0.035	0.062	0.085	0.040
	0	0.111	0.033	0.063	0.084	0.045
	0.5	0.088	0.033	0.047	0.065	0.035

Table 3 Values of feature space measure for lognormal data (location difference).

% Outliers	ρ	Gaussian	w_1	w_2	w_3	w_4
0	-0.1	0.365	0.263	0.282	0.283	0.272
	0	0.401	0.287	0.311	0.324	0.305
	0.5	0.507	0.409	0.421	0.462	0.422
10	-0.1	0.607	0.517	0.441	0.609	0.509
	0	0.649	0.512	0.477	0.648	0.526
	0.5	0.836	0.614	0.593	0.792	0.644
20	-0.1	0.757	0.555	0.495	0.736	0.603
	0	0.834	0.548	0.504	0.780	0.576
	0.5	1.022	0.668	0.616	0.927	0.680
30	-0.1	0.733	0.476	0.482	0.551	0.511
	0	0.749	0.533	0.496	0.558	0.539
	0.5	0.995	0.668	0.605	0.744	0.675

The values of the feature space measure (*FSM*) proposed by Nguyen and Ho (2008) appear in Table 3. A small value is desirable and the minimum for each configuration appears in bold face. Here the kernel with weight w_2 has the minimum value in 8 cases, while the kernel with w_1 achieves the minimum in the remaining cases. The Gaussian kernel has the largest value of this criterion in all but one case. The weighted kernels are successful in reducing the total within class standard deviation *in the direction between the class centres* relative to the between group distance. As mentioned in Section 3, within class variation in the direction between the class centres will specifically be detrimental to classification performance, and kernels that reduce this variation, can be expected to perform well, as is the case for the weighted kernels in this example. Similar results were obtained for the other configurations considered in this paper.

Configuration 2

We consider normal data with location difference. We used $p_r = 3$ relevant variables and $p_n = 7$ noise variables. For the relevant variables we used $\mu_1 = \mathbf{0}$, $\mu_2 = 2\mathbf{e}_3$. The groups had equal covariance matrices: $\Sigma_1 = \Sigma_2 = \Sigma^{(\rho)}$. Data without outliers as well as data with 10%, 20% and 30% outliers were considered. For the outliers $\mu_1 = 4\mathbf{e}_3$, $\mu_2 = -2\mathbf{e}_3$ and $\Sigma_1 = \Sigma_2 = \Sigma^{(\rho)}$ were used. The noise variables were generated with zero means and variances equal to 10, and were uncorrelated with all other variables. Sample sizes (including outliers if present) of $n_1 = n_2 = 25$ were used. Table 4 contains the results.

The performance of the Gaussian kernel is again improved by using the proposed weights. None of the four different weighted kernels consistently performs the best. For noisy data without outliers or with only 10% outliers (the first six rows of the table), the kernel with weight w_3 performed the best, but when the outlier proportion was increased, the kernels with weights w_1 and w_4 yielded the best error rates. Differences between the performance of w_1 and w_4 are small.

Table 4 Mean and standard error of test errors for normal data (location difference).

% Outliers	ρ	Gaussian	w_1	w_2	w_3	w_4
0	-0.1	.077(.0027)	.065(.0020)	.091(.0033)	.054 (.0017)	.064(.0025)
	0	.109(.0033)	.083(.0020)	.110(.0035)	.075 (.0021)	.088(.0028)
	0.5	.171(.0040)	.129(.0019)	.158(.0042)	.124 (.0024)	.135(.0021)
10	-0.1	.146(.0045)	.101(.0034)	.133(.0045)	.095 (.0033)	.106(.0037)
	0	.169(.0042)	.121(.0041)	.157(.0058)	.112 (.0036)	.122(.0040)
	0.5	.244(.0053)	.163 (.0039)	.188(.0053)	.163 (.0039)	.164(.0036)
20	-0.1	.232(.0054)	.150(.0044)	.187(.0066)	.159(.0047)	.147 (.0047)
	0	.248(.0060)	.151(.0043)	.187(.0064)	.167(.0046)	.148 (.0044)
	0.5	.305(.0048)	.179(.0047)	.203(.0065)	.214(.0050)	.178 (.0053)
30	-0.1	.325(.0052)	.233 (.0049)	.248(.0057)	.250(.0049)	.239(.0048)
	0	.342(.0051)	.246 (.0044)	.262(.0054)	.268(.0053)	.255(.0042)
	0.5	.387(.0045)	.275 (.0058)	.302(.0067)	.310(.0053)	.280(.0056)

Configuration 3

We again consider normal data with a location difference. We used $p_r = 2$ relevant variables, $p_n = 3$ noise variables and sample sizes $n_1 = n_2 = 20$. For the relevant variables we again set $\mu_1 = \mathbf{0}$, but in the second group we took $\mu_2 = -3\mathbf{e}_2$ for 10 of the data points and $\mu_2 = 3\mathbf{e}_2$ for the remaining 10 data points. This is clearly a scenario where a linear classifier will not perform well. Data without outliers were considered, as well as data with 10%, 20% and 30% outliers (for which we took $\mu_1 = [3, -3]$) only in the first group. The same covariance structure as in the previous configuration was used, and the noise variables were also generated in the same way as before.

As seen in Table 5, the weighted kernels again perform better than the Gaussian. The kernels with weights w_1 and w_4 yielded the best error rates, the differences between their performance again being small.

Table 5 Mean and standard error of test errors for normal data (location difference).

% Outliers		ρ	Gaussian	w_1	w_2	w_3	w_4
0	—	−0.1	.111(.0034)	.063 (.0022)	.085(.0041)	.069(.0026)	.067(.0021)
		0	.111(.0036)	.067 (.0025)	.092(.0046)	.073(.0029)	.071(.0023)
		0.5	.165(.0042)	.091 (.0022)	.115(.0042)	.105(.0038)	.094(.0020)
10	—	−0.1	.111(.0046)	.076 (.0028)	.100(.0055)	.087(.0038)	.076 (.0030)
		0	.121(.0044)	.078 (.0020)	.103(.0049)	.093(.0037)	.079(.0024)
		0.5	.174(.0044)	.099 (.0026)	.124(.0049)	.123(.0043)	.104(.0028)
20	—	−0.1	.115(.0038)	.092(.0026)	.111(.0061)	.093(.0040)	.089 (.0031)
		0	.123(.0042)	.090(.0027)	.121(.0053)	.100(.0032)	.088 (.0030)
		0.5	.175(.0043)	.113 (.0024)	.143(.0053)	.128(.0038)	.122(.0030)
30	—	−0.1	.123(.0042)	.091(.0029)	.112(.0053)	.108(.0046)	.084 (.0036)
		0	.132(.0039)	.095(.0025)	.123(.0058)	.112(.0046)	.092 (.0029)
		0.5	.178(.0043)	.119 (.0030)	.147(.0061)	.142(.0054)	.121(.0034)

Configuration 4

We consider normal variables with difference in spread. We used $p_r = 3$ relevant variables, $p_n = 7$ noise variables and samples of sizes $n_1 = n_2 = 25$. We set $\mu_1 = \mu_2 = \mathbf{0}$, resulting in the groups having the same location. For the relevant variables we used $\Sigma_1 = \Sigma^{(\rho)}$ and $\Sigma_2 = 5\Sigma^{(\rho)}$. For outliers we took $\mu_1 = \mu_2 = \mathbf{0}$, $\Sigma_1 = 7\Sigma^{(\rho)}$ and $\Sigma_2 = 0.1\Sigma^{(\rho)}$. Noise variables were generated in the same way as before. The results appear in Table 6. Here w_1 yielded the lowest error rates when there were no outliers, but w_4 performed best when outliers were present.

Table 6 Mean and standard error of test errors for normal data (spread difference).

% Outliers		ρ	Gaussian	w_1	w_2	w_3	w_4
0	−0.1		.259(.0038)	.107 (.0024)	.124(.0041)	.132(.0028)	.110(.0025)
	0		.255(.0030)	.104 (.0018)	.115(.0031)	.134(.0028)	.108(.0022)
	0.5		.268(.0031)	.125 (.0022)	.133(.0039)	.159(.0030)	.128(.0024)
10	−0.1		.318(.0042)	.140(.0039)	.145(.0060)	.226(.0045)	.129 (.0039)
	0		.313(.0034)	.148(.0043)	.146(.0062)	.220(.0039)	.137 (.0042)
	0.5		.338(.0047)	.153(.0039)	.147(.0047)	.245(.0046)	.148 (.0040)
20	−0.1		.349(.0042)	.170(.0055)	.152(.0071)	.273(.0056)	.140 (.0049)
	0		.358(.0055)	.176(.0071)	.166(.0098)	.275(.0063)	.138 (.0061)
	0.5		.370(.0057)	.190(.0063)	.185(.0100)	.286(.0055)	.164 (.0063)
30	−0.1		.419(.0078)	.370(.0066)	.396(.0089)	.398(.0073)	.356 (.0092)
	0		.416(.0073)	.380(.0076)	.428(.0079)	.408(.0070)	.378 (.0090)
	0.5		.430(.0073)	.388(.0071)	.436(.0073)	.409(.0069)	.386 (.0087)

We also studied this same configuration without outliers or noise variables ($p_r = 3$ and $p_n = 0$). The results appear in Table 7.

Table 7 Mean and standard error of test errors for normal data (location difference). (No noise variables or outliers)

% Outliers		ρ	Gaussian	w_1	w_2	w_3	w_4
0	−0.1		.071 (.0030)	.077(.0020)	.088(.0036)	.083(.0032)	.085(.0033)
	0		.068 (.0028)	.073(.0022)	.083(.0035)	.083(.0034)	.079(.0023)
	0.5		.081 (.0022)	.091(.0026)	.086(.0037)	.088(.0043)	.091(.0033)

In this case the Gaussian kernel results in lower error rates than the weighted kernels. However, the differences are of a much smaller magnitude than the differences between the weighted kernels and the Gaussian kernel when outliers and/or noise are present.

Configuration 5

We considered lognormal data with spread difference. We used the following transformation to generate multivariate lognormal observations having covariance matrix $\Sigma^{(\rho)}$ (cf. Johnson, 1987). Let $\mathbf{Z} = [Z_1, Z_2, \dots, Z_p]'$ be a p -vector having a multivariate normal distribution with mean vector $\mathbf{0}$ and covariance matrix $\Sigma^{(\nu)}$. If we take $\nu = \log[\rho(e-1) + 1]$, put $Y_i = e^{Z_i}$ and $X_i = \frac{Y_i}{\sqrt{e(e-1)}} - \frac{1}{\sqrt{e-1}}$ ($i = 1, 2, \dots, p$), the p -vector $\mathbf{X} = [X_1, X_2, \dots, X_p]'$ has mean vector $\mathbf{0}$ and covariance matrix $\Sigma^{(\rho)}$, while $X_1, X_2, \dots, X_p$ are lognormally distributed. We used $\Sigma_1 = \Sigma^{(\rho)}$ and $\Sigma_2 = 2\Sigma^{(\rho)}$ and for outliers we took $\Sigma_1 = 9\Sigma^{(\rho)}$ and $\Sigma_2 = 0.1\Sigma^{(\rho)}$. We used $n_1 = n_2 = 25$, $p_r = 5$ and $p_n = 0$. The results appear in Table 8.

The relative performance of the methods is similar to the previous example, but the kernels with weights w_2 and w_3 yielded the lowest error rate in one case each. Once again the weighted kernels outperform the Gaussian kernel even when there are no outliers or noise variables.

Table 8 Mean and standard error of test errors for lognormal data (spread difference).

% Outliers		ρ	Gaussian	w_1	w_2	w_3	w_4
0	−0.1		.179(.0068)	.078(.0062)	.121(.0071)	.062 (.0054)	.077(.0060)
	0		.122(.0045)	.005(.0005)	.012(.0012)	.030(.0028)	.004 (.0004)
	0.5		.195(.0033)	.141 (.0030)	.147(.0032)	.168(.0033)	.144(.0030)
10	−0.1		.277(.0092)	.162(.0069)	.189(.0071)	.174(.0056)	.159 (.0066)
	0		.190(.0082)	.007 (.0005)	.017(.0011)	.075(.0049)	.008(.0007)
	0.5		.257(.0078)	.154 (.0031)	.157(.0034)	.189(.0040)	.154 (.0029)
20	−0.1		.424(.0091)	.230(.0085)	.234(.0078)	.241(.0060)	.226 (.0085)
	0		.375(.0101)	.013 (.0014)	.025(.0016)	.118(.0073)	.013 (.0009)
	0.5		.371(.0137)	.157(.0033)	.164(.0037)	.219(.0061)	.152 (.0032)
30	−0.1		.547(.0097)	.260(.0074)	.273(.0076)	.309(.0068)	.259 (.0074)
	0		.589(.0064)	.033 (.0033)	.046(.0030)	.261(.0067)	.044(.0043)
	0.5		.551(.0143)	.194(.0053)	.188 (.0050)	.267(.0093)	.193(.0064)

In addition to the 5 configurations, the configurations used in a study by Pavur (2002) were also considered. Pavur (2002) used data from bivariate normal populations without any outliers (configurations A and B) and with 10% outliers in each group (configurations C through K). The effect of the position of outliers on the performance of various classification techniques (such as linear and quadratic discriminant analysis, nearest neighbours and different mathematical programming techniques) were studied. Details of the configurations are given in Table 9.

Table 9 Bivariate normal configuration used in Pavur (2002).

Configuration	Population 1			Population 2				
	Non-outliers	Outliers		Non-outliers	Outliers			
	μ_{11}	Σ_{11}	μ_{12}	Σ_{12}	μ_{21}	Σ_{21}	μ_{22}	Σ_{22}
A	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$	—	—	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$	—	—
B	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 3 & 2.7 \\ 2.7 & 3 \end{bmatrix}$	—	—	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 3 & -0.3 \\ -0.3 & 3 \end{bmatrix}$	—	—
C	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0.75 \\ 0.75 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 \\ -20 \end{bmatrix}$	$\begin{bmatrix} 9 & 6.75 \\ 6.75 & 9 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 1 & 0.75 \\ 0.75 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 20 \end{bmatrix}$	$\begin{bmatrix} 9 & 6.75 \\ 6.75 & 9 \end{bmatrix}$
D	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 \\ -20 \end{bmatrix}$	$\begin{bmatrix} 0.2 & 0 \\ 0 & 9 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 20 \end{bmatrix}$	$\begin{bmatrix} 0.2 & 0 \\ 0 & 9 \end{bmatrix}$
E	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0.25 \\ 0.25 & 1 \end{bmatrix}$	$\begin{bmatrix} -5 \\ -5 \end{bmatrix}$	$\begin{bmatrix} 25 & 15 \\ 15 & 25 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0.75 \\ 0.75 & 1 \end{bmatrix}$	$\begin{bmatrix} -5 \\ -5 \end{bmatrix}$	$\begin{bmatrix} 25 & 10 \\ 10 & 25 \end{bmatrix}$
F	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1.5 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}$	$\begin{bmatrix} 25 & 0 \\ 0 & 25 \end{bmatrix}$
G	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -10 \\ -10 \end{bmatrix}$	$\begin{bmatrix} 25 & 0 \\ 0 & 25 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -10 \\ -10 \end{bmatrix}$	$\begin{bmatrix} 25 & 0 \\ 0 & 25 \end{bmatrix}$
H	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 3 \\ -3 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 3 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
I	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 3 \end{bmatrix}$	$\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ -3 \end{bmatrix}$	$\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$
J	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 13 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -10 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$
K	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -10 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$	$\begin{bmatrix} 3 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 13 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$

Our purpose in this paper is not to compare KFDA with the techniques in the study by Pavur (2002), but to compare the performance of the KFD classifiers using the weighted kernels to the KFD classifier using the Gaussian kernel for the different configurations.

We used training sample sizes $n_1 = n_2 = 20$ and the test dataset contained 1000 cases per group. Test errors were averaged over 100 Monte Carlo repetitions. The results are given in Table 10. The smallest average error rate for each configuration appears in bold face.

The Gaussian kernel performs best only in configuration B, which is one of the two configurations without outliers, while it performs the worst in 7 configurations. In 10 configurations, the best performer is one of the weighted kernels: the kernel with weight w_1 in 7 configurations and the kernel with weight w_3 in 2 configurations. The kernel with weight w_2 performs best in 1 configuration, but is the worst in 4 configurations. Although the kernel with weight w_4 never performs the best, it results in a lower error rate than the Gaussian kernel in 10 configurations and is never the worst performer. The weighted kernels are successful in achieving reduced error rates in the various outlier configurations considered by Pavur (2002).

Table 10 Mean and standard error of test errors for configurations A-K.

Configuration	Gaussian	w_1	w_2	w_3	w_4
A	.1249(.00161)	.1211(.00147)	.1253(.00158)	.1191 (.00135)	.1212(.00146)
B	.1737 (.00241)	.1741(.00223)	.1754(.00245)	.1742(.00244)	.1739(.00219)
C	.0998(.00218)	.0957 (.00196)	.1006(.00223)	.0972(.00189)	.0958(.00195)
D	.1894(.00198)	.1699 (.00141)	.1712(.00141)	.1780(.00196)	.1748(.00201)
E	.0518(.00139)	.0510(.00109)	.0502 (.00112)	.0506(.00116)	.0513(.00110)
F	.0833(.00181)	.0774 (.00119)	.0838(.00170)	.0781(.00126)	.0776(.00120)
G	.0733(.00120)	.0706 (.00097)	.0708(.00099)	.0712(.00098)	.0709(.00097)
H	.0815(.00167)	.0752 (.00108)	.0771(.00117)	.0775(.00124)	.0753(.00105)
I	.0835(.00176)	.0764 (.00116)	.0782(.00127)	.0798(.00147)	.0769(.00121)
J	.0854(.00154)	.0743 (.00108)	.0744(.00102)	.0752(.00123)	.0744(.00107)
K	.0906(.00207)	.0823(.00155)	.0903(.00216)	.0768 (.00112)	.0813(.00150)

5. Analysis of benchmark data sets

In addition to the Monte Carlo simulation studies, we compared the performance of the weighted kernels to that of the Gaussian kernel on seven benchmark data sets used by Rätsch et al. (2001). These data sets are available in the form of 100 splits into training and test sets at <http://ida.first.gmd.de/raetsch/data/benchmarks.htm>. The values δ and λ were determined for each of the weighted kernels by means of cross-validation, and a fixed value of $\gamma = 1/p$ was used. For the Gaussian kernel, γ and λ were determined by cross-validation. These values were then used to obtain the different classifiers, and the test sets were classified by each of the classifiers. The average error rates (and standard errors) appear in Table 11.

Table 11 Mean and standard error of test errors for benchmark data sets.

Data set	Gaussian	w_1	w_2	w_3	w_4
Heart	.1593(.0042)	.1579(.0045)	.1586(.0049)	.1557 (.0043)	.1584(.0046)
Cancer	.2618(.0047)	.2514(.0042)	.2513 (.0044)	.2535(.0042)	.2529(.0042)
Banana	.1066(.0005)	.1044(.0005)	.1077(.0005)	.1071(.0005)	.1041 (.0005)
German	.2373(.0022)	.2378(.0022)	.2388(.0021)	.2314 (.0023)	.2323(.0023)
Thyroid	.0421(.0018)	.0399(.0020)	.0389 (.0022)	.0420(.0020)	.0445(.0022)
Titanic	.2284(.0009)	.2222(.0012)	.2215 (.0012)	.2241(.0012)	.2222(.0013)
Diabetes	.2327(.0019)	.2308(.0016)	.2374(.0017)	.2321(.0017)	.2307 (.0016)

For each dataset, the smallest error rate appears in bold face. The Gaussian kernel never performs the best. The kernel with weight w_2 gives the best performance in 3 of the data sets, while the kernels with weights w_3 and w_4 each performs best twice. However, in most cases the differences are quite small.

6. Conclusion

The results of the simulation study and analysis of benchmark data sets indicate that the performance of the KFD classifier is improved when the proposed weighted kernels are used instead of the Gaussian kernel, especially when there were outliers and/or noise in the data. When comparing the four different weights, none of the proposed weighted kernels consistently perform best. However, in most cases the worst error rate achieved when using a weighted kernel is still lower than that obtained when using the Gaussian kernel.

Topics for further research include: using the proposed weighted kernels in other kernel techniques, such as support vector machines; using weights in other kernels, such as the polynomial kernel; investigating the effect of correlation structures where it is not assumed that all correlations between the relevant variables are necessarily equal; investigating the performance of the weighted kernels in configurations where the populations differ with respect to location as well as spread simultaneously and specifying the kernel hyperparameter by means of cross-validation or some other adaptive procedure.

Acknowledgement

The author would like to thank Sarel Steel for helpful suggestions and the referees for valuable comments which led to an improvement of the paper.

References

- BO, L.F., WANG, L AND JIAO, L.C. (2006). Feature Scaling for Kernel Fisher Discriminant Analysis Using Leave-One-Out Cross Validation. *Neural Computation* **18**, 961-978.
- CHEN, J-H. (2004). M-estimator based robust kernels for support vector machines. *Proceedings of the 17th International Conference on Pattern Recognition*, 168-171.
- CRISTIANINI, N., KANDOLA, J., ELISSEEFF, A. AND SHAWE-TAYLOR, J. (2002). On kernel-target alignment. In: T. Dietterich, S. Becker and D. Cohn, (Eds.), *Advances in Neural Information Processing Systems*, 14, MIT Press, Cambridge.
- GEMAN, S. AND McCLURE, D.E. (1987). Statistical methods for tomographic image reconstruction. *Bulletin of the International Statistical Institute*, **52**, 5-21.
- HERBRICH, R. AND WESTON, J. (1999). Adaptive margin support vector machines for classification. *Proceedings of the Ninth International Conference on Artificial Neural Networks* **2**, 880-885.
- HOLLAND, P.W. AND WELSCH, R.E. (1977). Robust regression using iteratively reweighted least- squares. *Communications in Statistics, Theory Methods*, **A6** 813-827.

- JOHNSON, M.E. (1987), *Multivariate Statistical Simulation*, Wiley & Sons Inc.
- KIM, S-J., MAGNANI, A. AND BOYD, S.P. (2006). Robust Fisher discriminant analysis. In *Advances in Neural Information Processing Systems* 659-666. MIT Press.
- LAMONT, M.M.C. (2008). Assessing the influence of observations on the generalisation performance of the kernel Fisher discriminant classifier. Unpublished PhD dissertation, Stellenbosch University.
- LI, Y., WESSELS, L.F.A., DE RIDDER, D. AND REINDERS, M.J.T. (2007). Classification in the presence of class noise using a probabilistic Kernel Fisher method. *Pattern Recognition* **40**, 3349 – 3357.
- LIAO, C.T. AND LAI, S.H. (2007). A robust kernel based on robust ρ -function. *Proceedings of the International Conference of Acoustics, Speech and Signal Processing*.
- LIANG, Z., ZHANG, D. AND SHI, P. (2006). Robust kernel discriminant analysis and its application to feature extraction and recognition. *Neurocomputing* **69**, 928–933.
- LOUW, N., AND LAMONT, M.M.C. (2009). Robust kernel Fisher discriminant analysis. *Proceedings of the International Conference on Robust Statistics*.

- MIKA, S., RÄTSCH, G., WESTON, J., SCHÖLKOPF, B. AND MÜLLER, K.-R. (1999). Fisher discriminant analysis with kernels. In Y.-H. Hu, J. Larsen, E. Wilson and S. Douglas (Eds.), *Neural Networks for Signal Processing*, **IX**, 41-48, IEEE Press, New York.
- MIKA, S., RÄTSCH, G. AND MÜLLER, K.-R. (2001). A mathematical programming approach to the kernel Fisher algorithm. In T.K. Leen, T.G. Dietterich and V. Tresp (Eds.), *Advances in Neural Information Processing Systems*, **13**, 591-597, MIT Press, London.
- NGUYEN, C.H. AND HO, C.B. (2008). An efficient kernel matrix evaluation measure. *Pattern Recognition* **41**, 3366 – 3372.
- OOSTHUIZEN, S. (2008). Variable selection for kernel methods with application to binary classification. Unpublished PhD dissertation, Stellenbosch University.
- PAVUR, R. (2002). A comparative study of the effect of the position of outliers on classical and nontraditional approaches to the two-group classification problem. *European Journal of Operational Research*, **136**, 603-615.
- RÄTSCH, G., ONODA, T. AND MÜLLER, K.-R. (2001). Soft margins for AdaBoost. *Machine Learning* **42**, 287-320.
- ROTH, V. (2006). Kernel Fisher Discriminants for Outlier Detection. *Neural Computation* **18**, 942–960.

- ROUSSEEUW, P.J. (1985). Multivariate estimation with high breakdown point. In: W. Grossmann et al., editors, *Mathematical Statistics and Applications*, Vol. B, 283–297, Reidel, Dordrecht.
- SCHÖLKOPF, B. AND SMOLA, A.J. (2002). *Learning with kernels: support vector machines, regularisation, optimisation, and beyond*. MIT Press, London.
- SONG, Q., HU, W. AND XIE, W. (2002). Robust support vector machine with bullet hole image classification. *IEEE Transactions on Systems, Man and Cybernetics* **32**, 440–448.
- TRAFALIS, T.B. AND GILBERT, R.C. (2005). Robust classification and regression using support vector machines. *European Journal of Operational Research* **173**, 893–909.
- WANG, N. AND RAFTERY, A.E. (2002). Nearest-Neighbor Variance Estimation (NNVE): Robust Covariance Estimation via Nearest-Neighbor Cleaning. *Journal of the American Statistical Association* **97**, 994–1006.
- WANG, W., XU, Z., LU, W., AND ZHANG, X. (2003). Determination of the spread parameter in the Gaussian kernel for classification and regression, *Neurocomputing* **55**, 643–663.
- XU, L., CRAMMER, K. AND SCHUURMANS, D. (2006). Robust support vector machine training via convex outlier ablation. *Proceedings of the 21st National Conference on Artificial Intelligence*.